

Buenos días, soy Ricardo Ruiz Fernández de Alba, y voy a presentar mi TFG: "Imputación de Datos Faltantes de Variables Meteorológicas y de Contaminación en Series Temporales Multivariantes IoT con Modelos Transformer"

La presentación se estructura en los siguientes seis puntos de acuerdo a la tipología de un TFG de Investigación: Introducción y Motivación, Estado del Arte, Modelos Transformers, Desarrollo experimental, Resultados y Discusión y Conclusiones.

En primer lugar, el contexto: Para estudiar la sostenibilidad ambiental de la región de Poqueira, en la Alpujarra Granadina, se tomaron datos de tráfico mediante 4 cámaras LPR (reconocimiento de matrículas) situadas en las localidades de Pampaneira, Bubión y Capileira y de variables meteorológicas y de contaminación con una unidad móvil equipada con sensores cerca de Pampaneira. Las cámaras junto con los sensores constituyen un sistema IoT que produce un dataset conformado por 114 variables, entre las cuales se encuentran CO, NO2, PM10 (partículas en suspensión procedentes de la combustión).

Al analizar los datos registrados cada hora de las 114 variables en dos períodos de 2023: invierno y verano, se encontró un 28.4% de datos faltantes, por diversos motivos tanto técnicos (fallos de los sensores) como ambientales (condiciones meteorológicas).

Por ejemplo, esta gráfica de PM10, una de 114 variables que conforman el dataset de este trabajo, representa una serie temporal parcialmente observada. Futuros análisis y predicciones sobre estas series serán más robustos si se completan de manera adecuada.

Frente a este problema, nos planteamos la siguiente pregunta de investigación: ¿En qué medida los diversos métodos de imputación pueden recuperar eficazmente los valores faltantes? ¿Cómo se posicionan los modelos Transformers en este contexto? Para responder a este interrogante, este trabajo plantea como objetivo principal el estudio de los modelos transformer para imputar series temporales.

Y como objetivos específicos:

1. Estudio del estado del arte.
 2. Fundamentación informática-matemática del Transformer.
 3. Elaboración de un Pipeline de Imputación y de un paquete de Python.
 4. Evaluación comparativa del rendimiento.
-

Así pues, comenzamos por presentar la forma general del problema de Imputación. Partimos de la Serie Temporal Original Parcialmente Observada X original. De esta serie, se elimina deliberadamente un porcentaje de valores observados produciendo X missing. Este proceso genera una máscara binaria M , con unos en la posiciones eliminadas (puntos rojos) y ceros en el resto.

Un método de imputación f se aplica a X missing y M , generando la serie temporal completa

Ahora bien, ¿Por qué eliminamos datos? Porque esto permite comparar los valores imputados con su valor verdadero (ground truth).

En la figura, se observan estos errores por exceso y por defecto.

Se hace uso de distintas métricas de error (Error absoluto medio, Error cuadrático medio, Error relativo medio).

Ninguna métrica es redundante, ya que cada una aporta información complementaria.

Continuamos con el estudio del estado del arte, estableciendo una taxonomía, que diferencia entre métodos estacionarios, métodos con información local, basados en matrices y de Deep Learning entre los que se encuentran los modelos Transformer.

Como métodos estacionarios, la media y la mediana pueden sustituir a los valores faltantes si se consideran invariantes en el tiempo.

Por otro lado, los métodos de información local estiman los valores faltantes basándose en los puntos adyacentes:

Para Forward Fill, el valor imputado $x_{sub\ t,j}$ gorro se imputa con $x_{sub\ t\ sub\ t_0,j}$ si el conjunto de los τ menores que t con posiciones 1 en la máscara es no vacío y 0 en caso contrario. T_0 es el máximo de este conjunto anteriormente descrito.

Análogamente con Backward Fill.

Esto significa que Forward Fill propaga el último valor observado hacia adelante, y Backward Fill rellena con el siguiente valor observado hacia atrás.

En el caso de la interpolación lineal, se tienen en cuenta ambos valores adyacentes. Estos se unen mediante una línea recta que determina los valores faltantes.

Como método basado en matrices, encontramos la Imputación Hankel .

La matriz de Hankel se define a partir una serie temporal, que se reorganiza en una matriz de tamaño $(n \text{ menos } k + 1) \times (kd)$, donde n : número de filas, k parámetro retardo que por defecto toma la parte entera de $n+1$ medios y d : número de variables.

La imputación se resuelve obteniendo la matriz de rango mínimo, para lo que se minimiza su norma nuclear, lo que permite obtener la serie temporal completa.

La arquitectura Transformer, en "Attention Is All You Need", es de los modelos más utilizados actualmente. Principalmente es conocido en el campo del procesamiento del lenguaje natural, aunque aquí lo utilizaremos para Series temporales.

Su diseño encoder-decoder utiliza N capas apiladas. El encoder procesa simultáneamente toda la secuencia de entrada generando una representación contextual que envía al decoder. Este focaliza su atención sobre tal representación generando una salida de forma autorregresiva. Tras una capa lineal final y Softmax se obtiene una distribución de probabilidad categórica sobre los posibles valores que pueden tomar las posiciones faltantes en la serie temporal.

En principio, la paralelización de la que hablábamos no respetaría el orden de la secuencia de embeddings si no fuera por el componente Positional encoding. Este utiliza funciones trigonométricas para proporcionar información de orden y posición.

Pero, sin lugar a duda, el núcleo del Transformer es su mecanismo de Atención, que toma como entrada las matrices de Consulta (Q), Claves (K) y Valores (V), construidas a partir de los embeddings con información posicional. La atención calcula la similitud entre Q y K mediante el producto escalar escalado y seguidamente la función Softmax obtiene los pesos de atención que ponderan los valores. Técnicamente se ejecutan múltiples mecanismos de atención en paralelo (uno por cabeza), que se concatenan en los componentes Multihead-Attention.

saits es un Transformer Modificado, que emplea una arquitectura especializada en imputación de series temporales con dos bloques DMSA (Diagonal Masked Self-Attention) y un tercer bloque.

El primer bloque (en color verde) se asemeja al encoder, mientras que el segundo (en color azul) al decoder. Entonces, ¿cuáles son las particularidades de este modelo respecto a Transformer?

Por un lado, el enmascaramiento diagonal (en color naranja oscuro) y un tercer bloque de combinación ponderada (en color marrón claro)

¿En qué consiste este enmascaramiento diagonal?

Pues que mientras que en Transformer cada punto puede atender a todos los demás, y a sí mismo, en saits la máscara diagonal bloquea la auto-atención, asignando menos infinito a los elementos de la diagonal principal. Tras aplicar Softmax los pesos diagonales se anulan, lo que significa que, al multiplicar por la matriz de valores, V , cada punto sólo atiende a los demás.

El procesamiento matemático de saits se desarrolla en tres etapas:

El primer bloque DMSA produce tras su última capa lineal, la salida $X1$. Una primera imputación a partir de esta se toma como entrada para el segundo bloque, que tras una función de activación ReLu entre dos capas lineales, produce $X2$.

El tercer bloque efectúa una combinación ponderada de $X1$ y $X2$, dando lugar a $X3$.

Por último, utilizando la serie original X , la salida del bloque combinado $X3$ y la máscara de datos faltantes M , obtenemos la serie temporal completa X gorro.

Este trabajo ha utilizado como herramienta la biblioteca PyPOTS, que desarrollaron los autores de saits en 2023.

Ahora bien, se ha elaborado una paquete de Python pampaneira_imputation para integrar las implementaciones de PyPOTS con los diversos métodos definidos en la taxonomía.

Este paquete consta de 4 módulos dedicados a: la carga, preprocesamiento, imputación y evaluación, junto con dos módulos para utilidades y configuraciones.

Este paquete figura en el repositorio indicado con licencia de código abierto, para hacerlo reproducible. Se han seguido las buenas prácticas de desarrollo de software mediante la metodología ágil y el desarrollo de test unitarios.

Utilizando los módulos del paquete desarrollado, se elaboró un Pipeline o Flujo de Trabajo, usando la Plataforma Jupiter que va ejecutando de manera secuencial las funciones de los módulos citados.

Destacamos la utilización de bibliotecas usuales del ecosistema de Python como Pandas, Numpy o Matplotlib.

Dentro del preprocesamiento, destacamos que han escalado usando Z-score a partir de entrenamiento, a validación y testing.

Por otro lado, la representación de la serie temporal se cambia para adaptarla al modelo de Deep Learning. En concreto, mediante un proceso de ventaneo deslizando con solapamiento: que las convierte en arrays tridimensionales

(número de ventanas x tamaño de ventana x número de características).

Recordemos el ejemplo original de la variable PM10 parcialmente observada.

Recordemos, en la gráfica superior, el ejemplo de la variable PM10 parcialmente observada.

Tras el flujo de Trabajo, podemos observar las gráficas con los valores imputados (línea verde de puntos) por saits:

Abajo a la izquierda con el patrón 'subseq': que elimina ráfagas de puntos. Y a la derecha con el patrón puntual 'point': que elimina puntos de forma aleatoria. La comparación con el ground truth (puntos rojos) proporciona la tabla de errores.

Esta es una de las 14 tablas de errores de evaluación que hay en la memoria. En concreto, la del período 1 con 10% de ausencia y patrón puntual.

Para visualizar mejor los resultados, se construyó este gráfico 3D. Observándose que: La interpolación lineal es el mejor de los métodos simples. Y que los modelos DL: Transformer & saits son superiores al resto de modelos. De hecho, saits mejora ligeramente a Transformer.

Se replicó el Pipeline con variaciones en los porcentajes y patrones de ausencia. Y se observó que, en la mayoría de los casos, conforme aumenta el porcentaje aumenta el error. Por otro lado, la ausencia con patrón ráfaga incrementa ligeramente el error respecto al mismo tanto por ciento puntual.

A la pregunta de investigación planteada inicialmente:

“¿En qué medida los métodos de imputación pueden recuperar eficazmente los valores faltantes? ¿Y Transformer?”

Podemos responder:

La eliminación artificial nos ha permitido comparar los valores imputados con los originales (ground truth) mediante métricas de evaluación. Transformer y saits son claramente superiores, según estas métricas, a los demás modelos. Además, se han estudiado sus arquitecturas en profundidad. Saits mejora ligeramente a Transformer. Aunque la ausencia por ráfaga da mayor error que la puntual, modela los fallos de los sensores de forma más realista.

Tras una planificación inicial, se elaboró el cronograma de tiempo real invertido en el trabajo dividido en tareas. Por otro lado, se añadió el presupuesto.

Ambos se encuentran desarrollados en el Capítulo 6 de la memoria.

El trabajo ha cumplido su objetivo principal: estudiar el uso de modelos Transformer para imputar valores faltantes en series temporales, demostrando su eficacia frente a métodos tradicionales.

- Se han alcanzado los objetivos específicos:
- Revisión del estado del arte.
- Fundamentos matemáticos e informáticos.
- Desarrollo de un pipeline y paquete en Python.
- Evaluación rigurosa del rendimiento.

Además, se ha hecho uso de conocimientos adquiridos en los grados, . El proyecto se ha alineado con principios éticos (datos abiertos y software libre) y con los Objetivos de Desarrollo Sostenible.

Gracias por su Attention.